



Università della Calabria

Corso di Laurea Magistrale in Ingegneria Informatica: Artificial
Intelligence and Machine Learning

Modelli e Tecniche per Big Data

Progetto 10: Analisi di Post Reddit per il Monitoraggio della Salute Mentale

Studente:

Luca Timpano

Matricola: 276570

Docenti:

Prof. Paolo Trunfio

Prof. Fabrizio Marozzo

Anno Accademico 2025/2026

Indice

1	Introduzione	1
1.1	Obiettivi del Progetto	1
1.2	Casi d'Uso	1
1.3	Tecnologie Utilizzate	1
1.4	Valutazione tecnica di progetti esistenti e documentazione	2
2	Analisi del dataset	3
2.1	Origine del dataset	3
2.2	Struttura e Classi	3
2.3	Pre-Processing e Data Cleaning	4
3	Architettura del Sistema	7
3.1	Progettazione Modulare	7
3.2	Flusso dei Dati	8
4	Elaborazione Dati con Apache Spark	9
4.1	Caricamento e Preparazione del DataFrame	9
4.2	Interrogazioni Aggregate e Analisi Descrittiva	10
4.3	Analisi Temporale dei Post	11
4.4	Analisi della Varianza e Feature Esplorative	13
4.5	Analisi psicolinguistica	14
5	Machine Learning e Intelligenza Artificiale	16
5.1	Estrazione delle Feature Importance	17
5.1.1	Regressione Logistica	17
5.2	Integrazione di un Large Language Model (LLM)	18
5.3	Latent Dirichlet Allocation (LDA)	20
6	Interfaccia Utente e Visualizzazione	23
6.1	Dashboard di Overview	24
6.1.1	get_eda_features() && get_label_variance()	24

6.1.2	get_top_users()	25
6.1.3	get_selfreference_analysis()	25
6.2	Analisi Temporale (Temporal Analysis)	25
6.2.1	correlazione_orario_etichetta()	26
6.2.2	get_eda_features()	26
6.2.3	get_posts_over_time()	26
6.3	Strumenti di Ricerca	27
6.4	Sezione Intelligenza Artificiale & Machine Learning	28
6.4.1	get_feature_importance_data()	28
6.4.2	generate_LLM_response()	29
6.4.3	perform_topic_modeling_LDA()	30
6.5	Streaming Dati	30
6.5.1	Simulatore di Flusso	31
7	Spark come sistema distribuito	32
8	Conclusioni	35
8.1	Valutazione Tecnica	35
8.2	Sviluppi Futuri	35

Capitolo 1

Introduzione

L'esponenziale crescita dei social media ha trasformato radicalmente la società moderna e le nostre modalità di interazione. Poiché gran parte delle relazioni si è spostata su queste piattaforme, tali strumenti possono essere considerati dei veri e propri sensori sociali. In questo panorama, **Reddit** si distingue per la sua struttura basata su comunità tematiche (subreddit), particolarmente preziose quando dedicate a temi delicati come la **salute mentale**.

1.1 Obiettivi del Progetto

Il seguente progetto mira a analizzare l'enorme quantità di dati generati dai social media, in particolare Reddit, per estrapolarne informazioni utili allo screening di malattie mentali, come la depressione e l'anoressia.

1.2 Casi d'Uso

Il sistema sviluppato mira a trovare applicazioni in casi di analisi esplorativa dei dati e supporto decisionale. Un secondo caso d'uso consiste nell'analisi semantica dei contenuti testuali, a supporto di studi preliminari nel campo della salute mentale digitale.

1.3 Tecnologie Utilizzate

Il progetto è stato sviluppato utilizzando un insieme di tecnologie principalmente orientate all'analisi di grandi quantità di dati. Il linguaggio di programmazione utilizzato è **Python**, in quanto flessibile e ricco di librerie dedicate al Machine Learning. Per l'elaborazione dati è stato utilizzato **Apache Spark**, sfruttando in

particolare il modulo **Spark SQL** e **Spark MLib** per la costruzione di pipeline di NLP (Natural Language Processing). I risultati ottenuti e l'interazione con l'utente sono state realizzate mediante **Streamlit**, un framework open source, che consente la creazioni di interfacce web dinamiche, in modo semplice e intuitivo. Infine, il sistema integra anche le API di **Google Gemini**, un LLM, in particolare il modello **gemini-3-flash-preview**, impiegato come assistente alla generazione di query personalizzate sul dataset.

1.4 Valutazione tecnica di progetti esistenti e documentazione

La fase di progettazione del sistema è stata preceduta da uno studio di soluzioni analoghe presenti. I principali riferimenti analizzati sono:

- **Reddit Analysis** (Link): Per l'analisi esplorativa del dataset.
- **Reddit Streaming Pipeline** (Link) e **Realtime Reddit Pipeline** (Link): Per le esecuzione di altre query, l'implementazione di Spark Stream e l'uso del framework **streamlit** per la dashboard interattiva.

Per lo sviluppo dell'interfaccia di monitoraggio e la visualizzazione , sono stati consultate le seguenti documentazioni:

- **Streamlit** (Link): Adottato per la realizzazione della dashboard, il framework permette un'integrazione efficiente con la **SparkSession** e una gestione ottimizzata delle risorse attraverso meccanismi di caching.
- **Plotly Express** (Link): Utilizzato per la generazione di grafici interattivi.

Capitolo 2

Analisi del dataset

2.1 Origine del dataset

Il dataset utilizzato è eRisk 2018, composto da dati provenienti dalla piattaforma reddit, stilato per l'analisi di segnali di disagio psicologico attraverso l'analisi dei contenuti testuali pubblicata dagli utenti. Il dataset contiene post di utenti affetti da depressione e utenti con disturbi alimentari, è presente anche la classe di controllo senza evidenti patologie.

I dati forniti presentano formato XML, suddivisi per utente, con struttura annidata composta da: titolo, testo e metadati.

2.2 Struttura e Classi

Il dataset fornito è originariamente suddiviso in due sotto-dataset: uno contenente i pazienti affetti da depressione e uno relativo agli utenti con disturbi alimentari. I due dataset presentano la medesima struttura XML, questo ha semplificato la fase di pre-elaborazione dei dati, permettendo di unificare i due dataset senza particolari trasformazioni. Ogni dataset è diviso a sua volta in dieci chunk, ognuno contenente i file XML dei singoli individui. Per ogni utente sono presenti dieci post, scritti in periodi differenti. Segue un esempio di file XML:

```
<INDIVIDUAL>
  <ID>subject3</ID>

  <WRITING>
    <TITLE> </TITLE>
    <DATE> 2016-02-22 12:16:32 </DATE>
    <INFO> reddit post </INFO>
```

```

    <TEXT> .99 number looks smaller. For example, 0.99 looks
    ↪ cheaper in price compared to USD 1. </TEXT>
</WRITING>

<WRITING>
  <TITLE> Samsung Galaxy S7 edge vs Apple iPhone 6s Plus:
  ↪ first look </TITLE>
  <DATE> 2016-02-22 12:14:22 </DATE>
  <INFO> reddit post </INFO>
  <TEXT> </TEXT>
</WRITING>

<WRITING>
  <TITLE> Titolo Post 3 </TITLE>
  <DATE> 2016-02-22 12:10:00 </DATE>
  <INFO> reddit post </INFO>
  <TEXT> Contenuto testuale... </TEXT>
</WRITING>

<WRITING>
  <TITLE> Titolo Post 10 </TITLE>
  <DATE> 2016-02-22 11:00:00 </DATE>
  <INFO> reddit post </INFO>
  <TEXT> Ultimo post della sequenza. </TEXT>
</WRITING>
</INDIVIDUAL>

```

Ogni utente è associata a una label binaria nel dataset generale. Per unificare i dataset è stato quindi necessario rimappare le etichette nel seguente modo:

- 0: Control (utenti sani)
- 1: Depression
- 2: Anorexia

2.3 Pre-Processing e Data Cleaning

Per rendere il dataset più facilmente esaminabile e più adatto dall'elaborazione distribuita con Apache Spark il dataset è stato convertito in un formato tabellare CSV.

```

import xml.etree.ElementTree as ET

# Caricamento del file
tree = ET.parse(full_path)
root = tree.getroot()

# Estrazione ID del soggetto
id_ = root.findtext("ID", default="").strip()

# Iterazione sui 10 elementi WRITING
for writings in root.findall("WRITING"):
    title = writings.findtext("TITLE", default="")
    date = writings.findtext("DATE", default="")
    text = writings.findtext("TEXT", default="")

    # Esempio di utilizzo dati
    print(f"ID: {id_} | Data: {date} | Titolo: {title}")

```

Parallelamente si sono rimossi tutti quei elementi che potevano andare a rappresentare rumore per eventuali algoritmi NLP come i **link**:

```

import re

def remove_urls(text):
    if not text:
        return ""
    url_pattern = re.compile(r'http\S+|www\S+|https\S+',
        ↪ re.IGNORECASE)
    return url_pattern.sub('', text)

```

Un'ulteriore criticità dei file XML era l'assenza delle etichette (label) al loro interno; queste erano infatti memorizzate in un file testuale esterno con struttura ID LABEL. È stato quindi necessario mappare le etichette per ID per ogni dataset e utente.

```

def load_labels(path):
    labels = {}
    with open(path, "r", encoding="utf-8") as f:
        for line in f:
            user_id, label = line.strip().split()
            labels[user_id] = label
    return labels

```


Di seguito invece viene riportato il codice che rimappa le etichette:

```
    if task_type == 'depression':  
        final_label = int(raw_label) # 0 o 1  
elif task_type == 'anorexia':  
    final_label = 2 if int(raw_label) == 1 else 0
```

Infine la conversione da dataframe a csv:

```
df = pd.DataFrame(data, columns=[  
    "id", "title", "date", "text", "info", "label"  
)  
  
df.to_csv(output_file, index=False, encoding="utf-8",  
    ↪ quoting=csv.QUOTE_ALL)
```

Capitolo 3

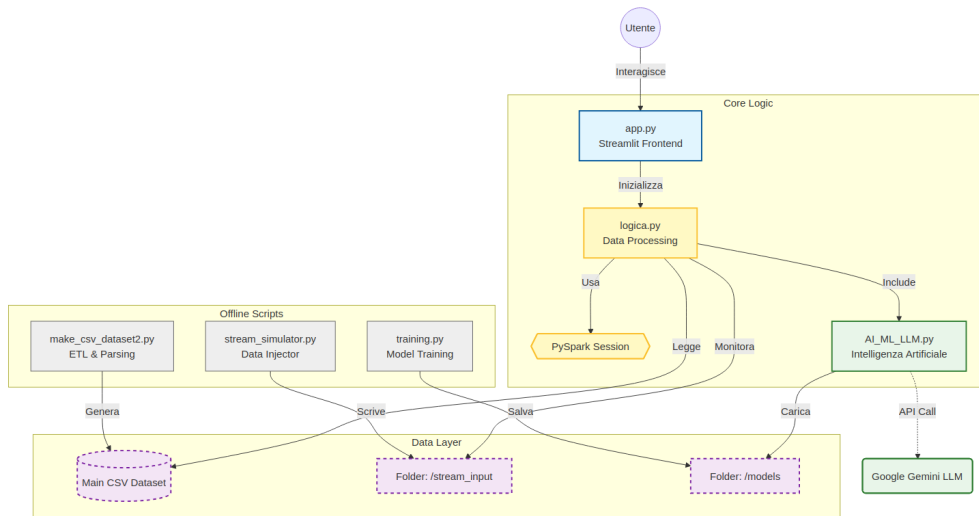
Architettura del Sistema

3.1 Progettazione Modulare

L'applicazione è stata pensata per avere architettura modulare, con una netta separazione della logica dalla rappresentazione delle query. Il progetto si divide in tre file principali:

- **Logica.py**: che include tutta la logica di elaborazione dei dati e le operazioni di analisi effettuate tramite Apache Spark, che verranno approfondite successivamente.
- **AI_ML_LLM.py**: è invece dedicato all'integrazione di tecniche di machine learning e alla definizione di un Large Language Model.
- **app.py**: implementa l'interfaccia grafica web tramite streamlit, gestendo la visualizzazione dei dati, l'interazione con l'utente e il coordinamento dei moduli di backend e di AI.

Questa separazione aumenta la manutenibilità del codice e la sua leggibilità, facilitando quindi interventi di estensione del sistema.



3.2 Flusso dei Dati

Il flusso dei dati inizia con il caricamento del dataset preprocessato all'interno di un Dataframe Spark, che viene mantenuto in memoria per ottimizzare tutte le operazioni. A partire da questo vengono poi effettuate trasformazioni di tipo narrow e wide, come il calcolo di statistiche aggregate, l'estrazione di feature testuali e l'analisi temporale dei post. I risultati delle elaborazioni Spark vengono successivamente ridotti e convertiti in Dataframe Pandas, limitandosi ai soli da da mostrare, al fine di evitare colli di bottiglia nella memoria locale. Tutti i dati vengono infine passati all'interfaccia Streamlit, che grazie all'uso della libreria Plotly, li renderizza sotto forma di grafici interattivi direttamente in una scheda web.



Capitolo 4

Elaborazione Dati con Apache Spark

Il sistema di elaborazione dati si basa sull'utilizzo di **Apache Spark**, utilizzato per gestire efficientemente il dataset eRisk2018. Come specificato nel capitolo precedente tutte le operazioni di elaborazione sono state implementate nel modulo `logica.py`, che incapsula l'accesso al DataFrame Spark e le principali query analitiche.

4.1 Caricamento e Preparazione del DataFrame

Il dataset, in formato CSV, viene caricato all'interno di un DataFrame Spark. Una fase cruciale è stata la fase di parsing per gestire al meglio campi multilinea e caratteri speciali. Inoltre per rendere più robusto il dataframe, ogni etichetta è stata convertita in intero e sono state eliminate le righe malformate o vuote.

```
def load_data(self):  
  
    self.df = self.spark.read \  
        .option("header", "true") \  
        .option("multiLine", "true") \  
        .option("sep", ",") \  
        .option("quote", "\"") \  
        .option("escape", "\\") \  
        .option("ignoreLeadingWhiteSpace", "true") \  
        .option("ignoreTrailingWhiteSpace", "true") \  
        .csv(self.path)  
  
    # Converto le label in interi
```

```

self.df = self.df.withColumn("label",
    ↪ col("label").cast(IntegerType()))

# Elimino le righe dove la label non è null
self.df = self.df.filter(col("label").isNotNull())

# Sostituisco eventuali valori nulli con ""
self.df = self.df.fillna("")

# Cache del DataFrame per migliorare le prestazioni
self.df.cache()
return self.df.count()

```

Per migliorare le prestazioni delle interrogazioni successive, il DataFrame risultante viene mantenuto in cache in memoria, riducendo il costo di ricalcolo delle trasformazioni più frequenti.

4.2 Interrogazioni Aggregate e Analisi Descrittiva

Una prima categoria di elaborazioni riguarda le **interrogazioni aggregate**, finalizzate a ottenere statistiche descrittive sul comportamento degli utenti. In particolare, viene calcolato il volume complessivo di testo prodotto da ciascun utente, sommando la lunghezza del titolo e del contenuto dei post e raggruppando i risultati per identificativo utente ed etichetta. Analogamente, viene calcolata la lunghezza media dei post per ciascuna classe, consentendo un confronto normalizzato tra le diverse categorie di utenti. Tali operazioni sfruttano le primitive di aggregazione di Spark (`groupBy`, `sum`, `avg`) e permettono di evidenziare differenze significative nel comportamento delle varie etichette. Segue la funzione `get_top_users(self, limit=30)` utilizzata per ottenere i top 30 utenti con la maggiore lunghezza totale del testo. Questa restituisce un DataFrame Pandas con gli utenti ordinati per lunghezza totale del testo.

```

def get_top_users(self, limit=30):
    # Creo una nuova colonna con la lunghezza del testo, raggruppo
    ↪ per id e label,
    # sommo le lunghezze e ordino in modo decrescente
    return self.df.withColumn("text_length", length(col("text")) +
    ↪ length(col("title"))) \
        .groupBy("id", "label") \

```

```

        .agg(sum("text_length").alias("total_length")) \
        .orderBy(desc("total_length")) \
        .limit(limit) \
        .toPandas()

```

Funzione `get_label_lenght_stats(self)`:

```

"""
Funzione per ottenere la media della lunghezza del testo per ogni
↪ etichetta.
Restituisce un DataFrame Pandas con la lunghezza media del testo
↪ per etichetta.
"""
def get_label_length_stats(self):
    # Calcolo la lunghezza del testo, raggruppo per etichetta,
    # calcolo la media e ordino alfabeticamente
    return self.df.withColumn("text_length", length(col("text")) +
        ↪ length(col("title"))) \
        .groupBy("label") \
        .agg(avg("text_length").alias("normalized_volume")) \
        .orderBy("label") \
        .toPandas()

```

4.3 Analisi Temporale dei Post

Il sistema implementa un'analisi temporale dei post per studiare la relazione tra orario di pubblicazione ed etichetta associata all'utente. Dalla colonna testuale contenente la data, viene effettuata una conversione sicura a timestamp, scartando i record non validi. Successivamente, viene estratta l'ora del giorno e calcolato il numero di post per ciascuna coppia etichetta-ora. Per tenere conto dello sbilanciamento tra le classi, i conteggi assoluti vengono normalizzati in percentuali tramite funzioni di Spark, ottenendo una distribuzione oraria comparabile tra i diversi gruppi. Questo tipo di analisi consente di individuare pattern circadiani potenzialmente distintivi tra utenti sani e utenti affetti da disturbi.

```

"""
Funzione per calcolare la correlazione tra l'orario di
↪ pubblicazione e l'etichetta.
Restituisce un DataFrame Pandas con la percentuale di post per
↪ ogni etichetta in ogni ora.
Ci si può aspettare che le diverse etichette mostrino pattern
↪ distinti in base all'ora del giorno.

```

```

"""
def correlazione_orario_etichetta(self):
    # filtriamo i record con date non nulle
    df_filtered = self.df.filter((col("date").isNotNull()) &
    ↪ (col("date") != ""))

    # estraiamo l'ora dalla data
    df_with_hour = df_filtered.withColumn("ts",
    ↪ try_to_timestamp(col("date"), lit("yyyy-MM-dd HH:mm:ss")))

    # rimuoviamo i record con timestamp nulli
    df_clean = df_with_hour.filter(col("ts").isNotNull())

    # estraiamo l'ora
    df_hour = df_clean.withColumn("hour", hour(col("ts")))

    # conto le occorrenze di ogni etichetta per ogni ora
    hourly_counts = df_hour.groupBy("label", "hour").count()

    # Per gestire lo sbilanciamento delle classi, invece di contare
    ↪ le occorrenze assolute,
    # calcoliamo la percentuale di post per ogni etichetta in ogni
    ↪ ora
    window_spec = Window.partitionBy("label")

    # normalizziamo i conteggi per ottenere le percentuali
    normalized_df = hourly_counts.withColumn(
        "percentage",
        (col("count") / sum("count").over(window_spec)) *
        ↪ 100).orderBy("hour")

    return normalized_df.toPandas()

```

Inoltre per evidenziare la mole crescente di dati generati dai social è stata effettuata una query che restituisce il numero di post scritti nel tempo:

```

def get_posts_over_time(self):
    """
    Restituisce il conteggio giornaliero dei post (Volume
    ↪ Totale).
    La base dati per il filtraggio temporale della dashboard.
    """

```

```

# Conversione stringa -> data e pulizia null
df_date = self.df.withColumn("ts",
    ↪ try_to_timestamp(col("date"), lit("yyyy-MM-dd
    ↪ HH:mm:ss"))) \
    .filter(col("ts").isNotNull()) \
    .withColumn("post_date",
    ↪ to_date(col("ts")))

# Aggregazione per singola data (Volume Totale aggregato)
return df_date.groupBy("post_date") \
    .agg(count("*").alias("post_count")) \
    .orderBy("post_date") \
    .toPandas()

```

4.4 Analisi della Varianza e Feature Esplorative

Oltre alle medie, viene analizzata la variabilità della lunghezza dei post all'interno di ciascuna classe tramite il calcolo della varianza. Questa misura permette di valutare la dispersione dei contenuti testuali e di identificare eventuali differenze nella consistenza dei post tra le categorie. A supporto di analisi esplorative (EDA), vengono inoltre estratte feature derivate, come la lunghezza totale del post e l'ora di pubblicazione, e selezionato un campione casuale di osservazioni, così da rendere più efficiente la visualizzazione dei dati senza compromettere la rappresentatività statistica.

```

"""
Funzione che mi estrapola le feature per applicare logiche
↪ esplorative
Restituisce un DataFrame Pandas con le feature estratte.
"""
def get_eda_features(self, sample_size=5000):
    # Creiamo le feature: lunghezza totale e ora del giorno
    df_features = self.df.withColumn("total_len",
    ↪ length(col("title")) + length(col("text"))) \
    .withColumn("ts",
    ↪ try_to_timestamp(col("date"),
    ↪ lit("yyyy-MM-dd HH:mm:ss"))) \
    .withColumn("hour", hour(col("ts")))

    # Filtriamo i null e prendiamo un campione (limite fissato a
    ↪ 5000 righe)

```



```

return df_features.filter(col("ts").isNotNull()) \
    .orderBy(rand()) \
    .select("label", "total_len", "hour") \
    .limit(sample_size) \
    .toPandas()

"""
Funzione per calcolare la varianza della lunghezza del testo per
↪ ogni etichetta
Restituisce un DataFrame Pandas con la varianza della lunghezza
↪ del testo per etichetta.
"""
def get_label_variance(self):
    return self.df.withColumn("text_len", length(col("text")) +
        ↪ length(col("title"))) \
        .groupBy("label") \
        .agg(variance("text_len").alias("variance")) \
        .orderBy("label") \
        .toPandas()

```

4.5 Analisi psicolinguistica

Studi clinici dimostrano che gli individui affetti da depressione o disturbi alimentari tendono a manifestare un'attenzione focalizzata sul sé, che si riflette linguisticamente in un uso significativamente più frequente di pronomi di prima persona singolare (**I, me, my, mine, myself**). La funzione utilizza espressioni regolari per isolare tali pronomi all'interno dei post unificati, calcolando il rapporto tra le occorrenze individuate e il numero totale di parole del testo. Pyspark aggrega questi dati per etichetta clinica, permettendo di osservare come la tendenza all'autoriferimento vari tra il gruppo di controllo e i sottogruppi affetti da patologie.

```

def get_self_reference_analysis(self):
    """
    Calcolo del rapporto di pronomi in prima persona singolare
    ↪ (I-usage).
    """
    # Regex per catturare i pronomi personali escludendo match
    ↪ parziali
    pronoun_regex = r"\b(i|me|my|myself|mine)\b"

    # Calcolo occorrenze pronomi e conteggio totale parole

```

```

df_analysis = self.df.withColumn("p_count",
    ↪ size(split(lower(col("text")), pronoun_regex)) - 1) \
    .withColumn("w_count",
    ↪ size(split(col("text"), " ")))

# Calcolo del ratio medio aggregato per classe (label)
result = df_analysis.filter(col("w_count") > 0) \
    .withColumn("ratio", col("p_count") /
    ↪ col("w_count")) \
    .groupBy("label").agg(avg("ratio") /
    .alias("avg_self_ref"))
return result.toPandas()

```

Capitolo 5

Machine Learning e Intelligenza Artificiale

L'analisi semantica del testo è stata realizzata mediante una pipeline di Natural Language Processing (NLP) basata su Spark ML. L'obiettivo è trasformare il testo grezzo dei post in una rappresentazione numerica adatta all'apprendimento automatico, mantenendo coerenza con il modello effettivamente addestrato nel file `training.py`. Per mantenere efficienza e velocità, lo script dedicato `training.py`, costruisce una pipeline Spark ML che viene memorizzata su disco. La pipeline segue i seguenti passaggi principali:

1. **Tokenizzazione:** viene utilizzato `RegexTokenizer` per estrarre esclusivamente parole alfabetiche con lunghezza minima di tre caratteri, rimuovendo rumore o token poco informativi.
2. **Rimozione delle stop-words:** tramite `StopWordsRemover` vengono eliminate sia le stop-word standard sia una lista personalizzata di termini frequenti ma semanticamente non rilevanti.
3. **Vettorizzazione TF-IDF:** il testo filtrato viene trasformato in vettori numerici mediante `CountVectorizer`.

```
# Tokenizzazione regex
tokenizer = RegexTokenizer(
    inputCol="text",
    outputCol="raw_words",
    pattern="[a-zA-Z]{3,}",
    gaps=False
)
```

```

# Rimozione stop-words
remover = StopWordsRemover(inputCol="raw_words",
    ↪ outputCol="filtered")

# TF-IDF
cv = CountVectorizer(inputCol="filtered", outputCol="raw_features",
    ↪ vocabSize=2000)
idf = IDF(inputCol="raw_features", outputCol="features")

```

5.1 Estrazione delle Feature Importance

Nel modulo `AI_ML_LLM.py` avviene a runtime l'estrazione delle feature importance attraverso un modello di **Regressione Logistica**. Il modello addestrato viene caricato da disco come `PipelineModel`. Questo approccio evita il riaddestramento del modello durante l'esecuzione dell'applicazione e riduce significativamente i costi computazionali. I coefficienti della regressione logistica rappresentano il peso associato a ciascun termine del vocabolario per ogni classe. Tali coefficienti vengono estratti direttamente dalla `coefficientMatrix` del modello multinomiale:

```

model = PipelineModel.load(model_path)
vocab = model.stages[3].vocabulary
lr_model = model.stages[-1]
coefficients = lr_model.coefficientMatrix.toArray()

```

Il risultato finale è un `DataFrame Pandas` che associa a ogni parola un peso numerico e la classe di riferimento, consentendo un'analisi interpretativa dei pattern linguistici caratteristici delle diverse condizioni cliniche.

5.1.1 Regressione Logistica

L'obiettivo della regressione logistica è trovare la relazione matematica tra una variabile dipendente (l'output Y) e una o più variabili indipendenti (le feature X). L'idea è quella di tracciare una linea retta che passi il più vicino possibile a tutti i punti del dataset. Questa retta rappresenta la "tendenza" dei dati. L'equazione della retta di regressione è:

$$Y = \beta_0 + \beta_1 X + \epsilon \quad (5.1)$$

Ogni post viene convertito in un vettore numerico x di dimensione 2000. La Regressione Logistica associa a ogni parola i del vocabolario un peso β_i . Per decidere se un testo appartiene a una classe (es. Anorexia), il modello calcola un

punteggio lineare z :

$$z = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \cdots + \beta_n x_n = \beta_0 + \sum_{i=1}^n \beta_i x_i \quad (5.2)$$

dove:

1. x_i è il valore numerico della parola estratto dal TF-IDF.
2. β_i è il coefficiente che l'algoritmo estrae dalla coefficientMatrix.

Per trasformare il punteggio z in una probabilità reale (compresa tra 0 e 1), il modello utilizza la funzione Softmax :

$$\sigma(\mathbf{z})_j = \frac{e^{z_j}}{\sum_{k=1}^K e^{z_k}} \quad (5.3)$$

Interpretazione dei coefficienti

- Se $\beta_i > 0$: La parola aumenta il valore di z , spingendo il modello a classificare il testo in quella categoria.
- Se $\beta_i < 0$: La parola diminuisce z , agendo come prova contraria per quella categoria.

5.2 Integrazione di un Large Language Model (LLM)

Il sistema integra un Large Language Model (LLM) tramite le API di Google Gemini, con l'obiettivo di fornire un assistente virtuale capace di supportare interrogazioni avanzate sui dati Spark. L'LLM è incapsulato nel modulo `AI_ML_LLM.py` e opera in stretta integrazione con Streamlit.

Dal punto di vista implementativo, l'LLM non accede direttamente ai dati grezzi, ma opera su una `SparkSession` già attiva e su `DataFrame` persistenti in memoria. I risultati intermedi e i modelli caricati (come la pipeline di regressione logistica) vengono mantenuti nella sessione applicativa, evitando ricaricamenti ridondanti e migliorando la reattività del sistema.

L'LLM riceve in input richieste in linguaggio naturale e genera codice PySpark. Il codice generato viene poi eseguito dinamicamente, permettendo:

- analisi esplorative personalizzate;
- aggregazioni statistiche su larga scala;

- visualizzazioni interattive tramite Streamlit.

Il tutto possibile attraverso il linguaggio naturale. Questa integrazione consente di combinare Big Data Analytics e intelligenza artificiale generativa, permettendo di automatizzare il processo di esplorazione dati. Segue un esempio indicativo della funzione utilizzata per effettuare la chiamata API.

```
@staticmethod
def generate_LLM_response(prompt):
    """
    Interfaccia core per la generazione di codice
    ↳ PySpark/Streamlit
    utilizzando il modello Gemini 3 Flash.
    """
    client = genai.Client(api_key=st.secrets["GEMINI_API_KEY"])

    context = """
    Sei un esperto Senior in PySpark e Streamlit.
    VARIABILI: 'df', 'spark', 'st', 'px', 'proc'.

    REGOLE:
    1. GESTIONE DATE: Usa `try_to_timestamp(col("date"),
    ↳ lit("yyyy-MM-dd HH:mm:ss"))`.
    2. FILTRO NULL: `.filter(col("nuova_colonna").isNotNull())`
    ↳ dopo ogni cast.
    3. PERFORMANCE: Aggrega sempre in Spark prima di chiamare
    ↳ .toPandas().
    4. OUTPUT: Solo codice Python puro all'interno di blocchi
    ↳ markdown.
    """

    try:
        response = client.models.generate_content(
            model="gemini-3-flash-preview",
            contents=f"{context}\n\nUser: {prompt}"
        )
        return response.text
    except Exception as e:
        return f"Errore critico: {str(e)}"
```

Il codice generato viene eseguito dinamicamente attraverso questa funzione nel modulo app.py.

```
def execute_spark_code(response_text, spark_env):
    """
    Estrae ed esegue dinamicamente il codice Python/Spark
    → generato.
    """
    # Estrazione del codice tramite Regular Expression
    code_match = re.search(r"```python\n(?:.*?)```", response_text,
        → re.DOTALL)

    if code_match:
        code = code_match.group(1)
        try:
            # Esecuzione nel contesto delle variabili locali
            → (spark, df, st)
            exec(code, {}, spark_env)
            return True, code
        except Exception as e:
            # Ritorna l'errore per il debugging in Streamlit
            return False, str(e)

    return False, "Nessun blocco di codice trovato nella risposta."
```

5.3 Latent Dirichlet Allocation (LDA)

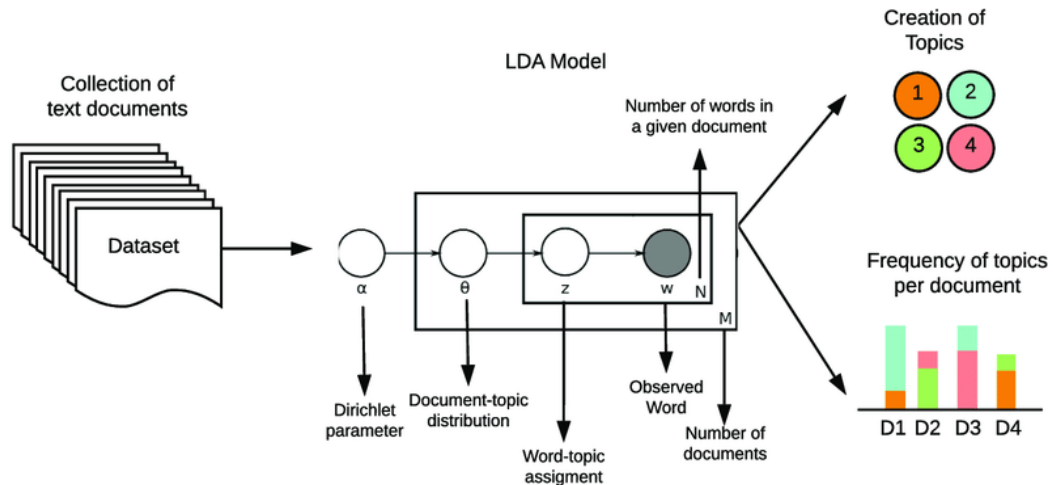
L'algoritmo Latent Dirichlet Allocation (LDA) è un modello probabilistico generativo utilizzato per scoprire la struttura tematica nascosta in un corpo di testi. Il funzionamento dell'algoritmo si basa su due assunti fondamentali:

- I documenti sono miscele di topic
- I topic sono miscele di parole

Il processo di addestramento esegue i seguenti passaggi

- L'algoritmo assegna casualmente ogni parola di ogni riga del dataset a uno dei k topic (dove k è il numero di cluster definito nello slider dell'interfaccia).
- L'algoritmo scansiona ripetutamente ogni parola e aggiorna l'assegnazione del topic basandosi su due criteri:
 1. Quanto è comune quel topic nella riga corrente?
 2. Quanto è comune quella parola in quel topic a livello globale?

- Dopo un numero prescelto di iterazioni (maxIter=20 nel nostro caso), le distribuzioni si stabilizzano. Il risultato finale è una matrice di probabilità che lega parole e temi.



Nel codice, l'algoritmo riceve in input un vettore di conteggi generato da `CountVectorizer`

$$P(\text{parola}|\text{documento}) = \sum_{\text{topic}} P(\text{parola}|\text{topic}) \cdot P(\text{topic}|\text{documento}) \quad (5.4)$$

```
def perform_topic_modeling_LDA(self, target_label=None,
    ↪ num_topics=5, num_words=10):

    # Pulizia dei dati ...

    cv = CountVectorizer(inputCol="filtered", outputCol="features",
    ↪ vocabSize=1000, minDF=2.0)
    lda = LDA(k=num_topics, maxIter=20)

    pipeline = Pipeline(stages=[tokenizer, remover, cv, lda])
    model = pipeline.fit(df_clean)

    # Estrazione termini e associazione al vocabolario
    vocab = model.stages[-2].vocabulary
    topics = model.stages[-1].describeTopics(num_words)

    topic_data = [{"topic_id": r['topic'], "terms": "",
    ↪ ".join([vocab[i] for i in r['termIndices']])} for r in
    ↪ topics.collect()]
```



```
return pd.DataFrame(topic_data)
```

Essendo però un modello non supervisionato i cluster formati non hanno etichetta, per risolvere si è deciso di inviare i gruppi di parole al modello Gemini 3 Flash. Questo passaggio trasforma la matrice dei termini, in etichette semantiche coerenti, facilitando la comprensione dei temi dominanti all'interno dei sottogruppi analizzati (Control, Depression, Anorexia).

```
@staticmethod
def name_topics_with_llm(topics_df):

    # Costruzione di una stringa di contesto leggibile per
    ↪ l'LLM
    topics_str = ""
    for index, row in topics_df.iterrows():
        topics_str += f"Topic {row['topic_id']}:
        ↪ {row['terms']}\n"

    prompt = "Prompt di contesto..."

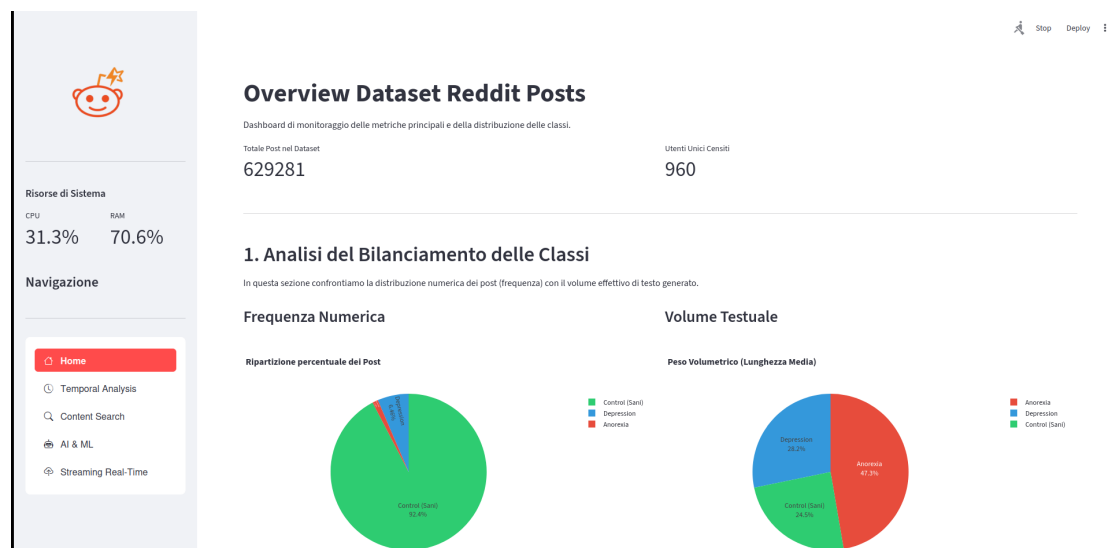
    try:
        # Inizializzazione client per richiesta specifica
        client =
        ↪ genai.Client(api_key=st.secrets["GEMINI_API_KEY"])

        response = client.models.generate_content(
            model="gemini-3-flash-preview",
            contents=prompt,
        )
        return response.text
    except Exception as e:
        return f"Errore generazione nomi: {e}"
```

Capitolo 6

Interfaccia Utente e Visualizzazione

L'interfaccia utente è stata progettata tramite il framework Streamlit, permettendo la navigazione tra i moduli di analisi statistica, ricerca semantica e monitoraggio in tempo reale. L'architettura sfrutta il decoratore `@st.cache_resource` per mantenere persistente la SparkSession, migliorando le prestazioni generali del sistema.



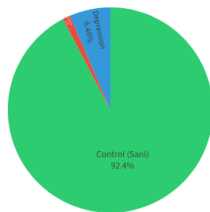
Per la visualizzazione grafica è stata utilizzata la libreria **Plotly Express**, che fornisce grafici interattivi.

6.1 Dashboard di Overview

La dashboard principale fornisce una sintesi immediata dello stato del dataset attraverso indicatori di performance analisi della distribuzione. L'obiettivo è identificare eventuali sbilanciamenti tra le classi analizzate: Control, Depression e Anorexia.

Frequenza Numerica

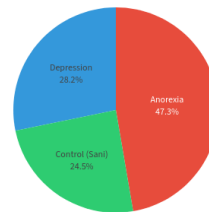
Ripartizione percentuale dei Post



Control (Sani)
Depression
Anorexia

Volume Testuale

Peso Volumetrico (Lunghezza Media)



Anorexia
Depression
Control (Sani)

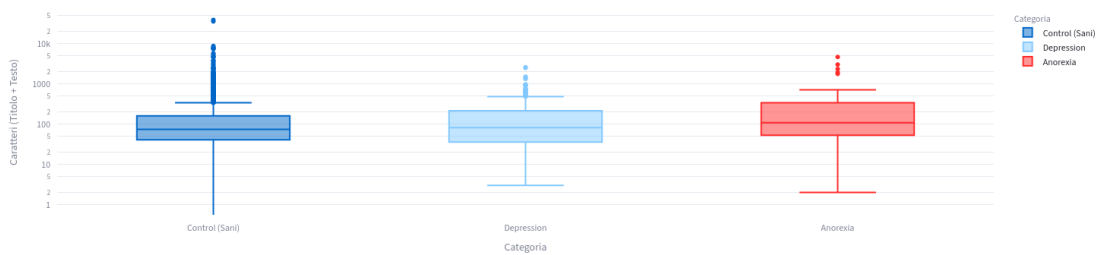
6.1.1 get_eda_features() && get_label_variance()

Viene poi implementato un Box Plot su scala logaritmica per analizzare la distribuzione della lunghezza dei post, permettendo di identificare visivamente la presenza di outlier e la varianza tra le diverse categorie.

2. Analisi della Distribuzione delle Lunghezze

Il seguente Box-Plot su scala logaritmica evidenzia la dispersione della lunghezza dei post per ogni categoria.

Distribuzione Lunghezza Post (Scala Log)



Indicatori di Varianza

La tabella a fianco mostra la varianza della lunghezza dei testi per classe. Una varianza elevata indica una minore omogeneità nello stile di scrittura degli utenti appartenenti a quella categoria.

	Categoria	Varianza (σ^2)
0	Control (Sani)	439,211.0093
1	Depression	172,513.9870
2	Anorexia	352,684.9746

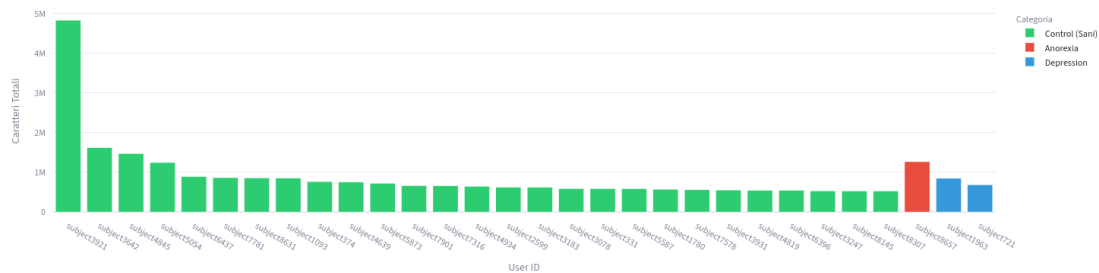
6.1.2 get_top_users()

Si è stampato il grafico che restituisce i trenta utenti che hanno scritto i post più lunghi.

3. Analisi Comportamentale Utenti

Identificazione degli utenti più attivi in termini di volume di caratteri prodotti (Top 30).

Ranking Utenti per Volume di Testo



6.1.3 get_selfreference_analysis()

Per l'analisi psicolinguistica si è implementato l'istogramma che mostra la tendenza degli utenti clinici all'autoriferimento.

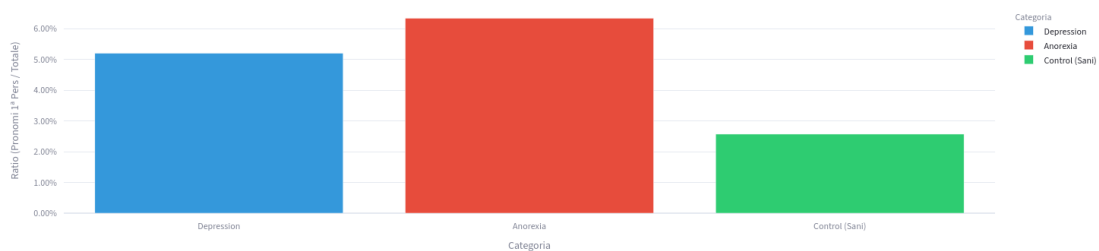
4. Approfondimento Linguistico

Indice di Focalizzazione Interna

In psicologia, un alto uso di self-reference è correlato a disturbi dell'umore.

Esegui Analisi Self-Reference

Self-Reference Ratio Medio per Categoria

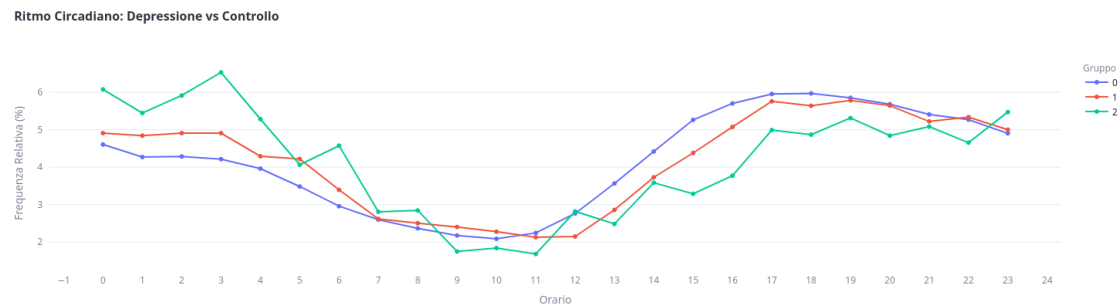


6.2 Analisi Temporale (Temporal Analysis)

Questo modulo investiga la dimensione cronologica dei dati per identificare pattern comportamentali legati ai cicli biologici e ai trend di pubblicazione.

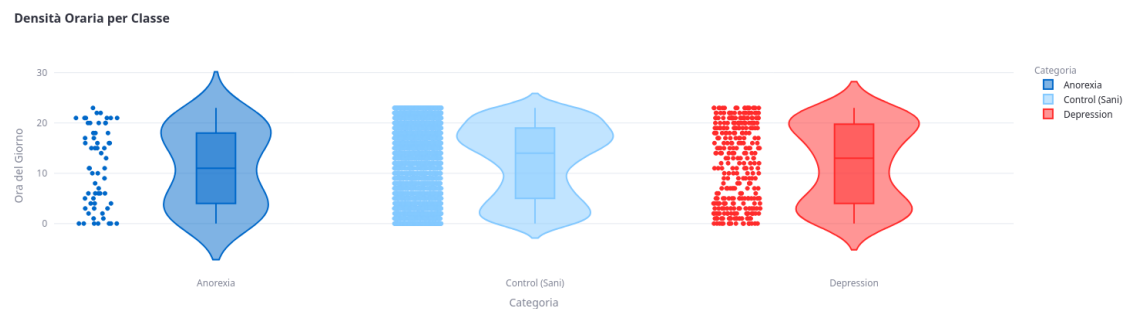
6.2.1 correlazione_orario_etichetta()

La funzione di aggregazione oraria calcola la frequenza relativa dei post per ogni ora del giorno. La visualizzazione grafica permette di evidenziare pattern comportamentali, si noti infatti un aumento dell'attività nelle ore notturne per le classi non control.



6.2.2 get_eda_features()

Per mostrare come la densità temporale si distribuisce nelle 24 ore, si è mostrato un Violin Plot.



6.2.3 get_posts_over_time()

Un grafico ad area con range slider permette di esplorare l'evoluzione del volume dei dati negli anni o mesi, fornendo la percentuale di crescita del volume di dati rispetto all'anno precedente.

Trend Temporale e Performance Metrics

Seleziona l'ampiezza della visualizzazione:

☐ Tutta la Timeline ☒ Anno Specifico ☐ Mese Specifico

Seleziona l'anno:

2017

Volume Post Selezionati

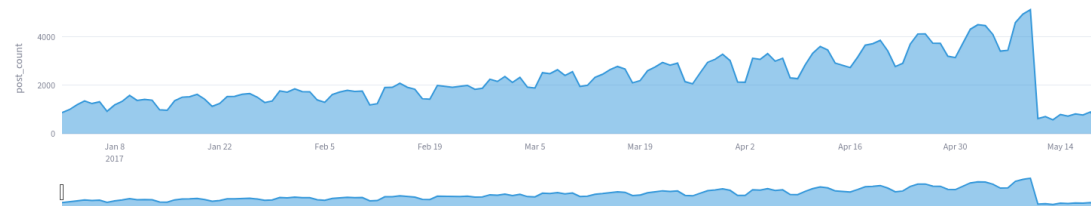
313,321

Rispetto all'anno precedente

Percentuale

↑ 60.10%

Analisi Volumetrica: Anno Specifico



6.3 Strumenti di Ricerca

La sezione di ricerca funge da interfaccia di interrogazione diretta per il cluster. La funzione di ricerca permette il filtraggio dinamico dei post tramite parole chiave. Il sistema invia la query a Spark, che restituisce un DataFrame filtrato limitato al numero di risultati selezionato dall'utente.

```
def search_keywords(self, keyword, limit):  
    kw = keyword.lower()      #ricerca case-sensitive  
  
    results = self.df.filter(  
        (lower(col("title")).contains(kw)) |  
        (lower(col("text")).contains(kw))  
    )  
  
    return results.select("id", "title", "date", "text",  
        ↪ "label").limit(limit).toPandas()
```

Esplorazione Contenuti

Ricerca Parole Chiave nei Post

Inserisci parola chiave: Limite risultati 50

Trovati 50 post rilevanti

id	Titolo	Data	Contenuto Post	Etichetta (0,1,2)
subject1010		2016-10-26 05:37:44	have you ever had pineapple and bacon on a cheeseburger? no pizza sauce, but there	0
subject1010		2016-10-26 06:09:21	that's fair. but perhaps it's a good stepping stone toward acceptance of pineapple on	0
subject1093		2017-05-10 12:35:44	hi pharmausi unfortunately your account is too young to post at this time, please wai	0
subject1093		2017-05-10 13:08:27	#### hi there akanet! regretably your submission has been removed as it did not fall	0
subject1093	/r/rnl weekly fitness challenge/fitness thread	2017-05-10 00:05:06	**step challenge** stridekick allows people from multiple fitness devices to compare	0
subject1093	daily support question megathread (may 10, 2017)	2017-05-10 08:33:20	##daily support thread hello! welcome to today's **daily support thread** this threa	0
subject1093		2017-05-10 12:55:39	#### hi there neb120! regretably your submission has been removed as it did not fal	0
subject1101		2016-09-10 17:56:12	male applebee's sure was.	0
subject1105	[h] \$30 apple store gift card [w] \$25 itunes/paypal	2016-08-05 00:00:50	would prefer itunes over paypal for this. rep profile:	0
subject1105		2016-07-30 00:28:11	apple store gift cards are exclusively for the apple store, sadly. i can't use it for other p	0

6.4 Sezione Intelligenza Artificiale & Machine Learning

In questa sezione viene raggruppata tutte le query realizzate grazie all'utilizzo di algoritmi di intelligenza artificiale e Large Language Model.

6.4.1 `get_feature_importance_data()`

L'interfaccia di analisi permette di esplorare la rilevanza semantica attraverso due modalità di visualizzazione principali:

1. Mappa Gerarchica
2. Grafico a barre

Analisi delle Feature

Distribuzione dei coefficienti del modello per la determinazione della rilevanza semantica.

Classe di riferimento:

2

Mappa Gerarchica Analisi Lineare



6.4.2 generate_LLM_response()

Nel seguente riquadro viene implementata una chat con un assistente virtuale capace di tradurre le query in linguaggio naturale in codice eseguibile da pySpark.

Assistente Tecnico

🗨️

mostrami i top 3 utente che hanno scritto i post più lunghi

Inserisci query di analisi...

↑

Reset Sessione Chat

Assistente Tecnico

🗨️

mostrami i top 3 utente che hanno scritto i post più lunghi

🔍

```

from pyspark.sql.functions import col, l

# 1. Pulizia dati: Filtriamo testi nulli
df_clean = df.filter(col("text").isNotNull)

# 2. Calcolo della lunghezza del testo
# Aggiungiamo una colonna con il numero
df_len = df_clean.withColumn("post_length", len(col("text")))

# 3. Aggregazione per utente (id)
# Cerchiamo la lunghezza massima raggiunta
# Nota: Aggregiamo prima di chiamare to
top_users_spark = df_len.groupBy("id") \
    .max("post_length") \
    .orderBy(desc("max_post_length")) \
    .limit(3)

df_clean.createTempView("df_clean")
top_users_spark.createTempView("top_users_spark")

df_clean.join(top_users_spark, on="id", how="inner")
            
```

Inserisci query di analisi...

↑

Reset Sessione Chat

Assistente Tecnico

Top 3 Utenti con i post più estesi

Inserisci query di analisi...

↑

Reset Sessione Chat

6.4.3 perform_topic_modeling_LDA()

Infine viene mostrata il risultato dell’algoritmo LDA, che mostra k gruppi di argomenti, dati in ingresso ad un LLM che ne genera i titoli:

Topic Modeling (Latent Dirichlet Allocation)

Estrazione non supervisionata di cluster tematici dal corpus testuale.

Configurazione Algoritmo

Sottogruppo Analitico:

Anorexia

Numero di Topic (K-Clusters)

4

Esegui Modellazione

Analisi Semantica Automatica:

0: Controllo corporeo e recupero 1: Disturbi alimentari e supporto 2: Conteggio calorie e pasti 3: Riflessioni e conversazione generale

Dettagli Matrice Termini

	topic_id	terms
0	0	eating, weight, control, food, hair, recovery, body, feel, years, much
1	1	eating, weight, feel, help, much, food, even, going, disorder, years
2	2	calories, breakfast, dinner, lunch, total, sugar, peanut, butter, snacks, much
3	3	also, well, love, doesn, pretty, though, right, something, work, looking

6.5 Streaming Dati

L’obiettivo di questo modulo è simulare l’arrivo continuo di nuovi dati e aggiornare istantaneamente le metriche di distribuzione delle classi senza dover riavviare l’elaborazione.

30

6.5.1 Simulatore di Flusso

Il simulatore agisce come la sorgente dei dati. Il suo compito è trasformare un dataset statico in un flusso dinamico. Il codice scrive piccoli file CSV (batch di 20 righe) in una cartella monitorata ogni 8 secondi nel nostro caso, ma il tempo di aggiornamento può essere deciso a priori in base allo scopo. Il nucleo dell'elaborazione è costituito da un'aggregazione continua che viene attivata da un trigger specifico ogni volta che un nuovo file CSV viene rilevato nella cartella di input. Spark processa i dati in modo sequenziale, eseguendo internamente un conteggio raggruppato per le diverse etichette di classe.

Live Data Stream (Spark Structured Streaming)

In attesa di dati dallo stream...

Live Data Stream (Spark Structured Streaming)

Distribuzione Classi in Tempo Reale



Ultimo aggiornamento: 2026-01-24 11:41:10.029398

Capitolo 7

Spark come sistema distribuito

Al fine di ottimizzare le prestazioni computazionali e simulare uno scenario di produzione per l'analisi di Big Data, l'applicativo è stato implementato sfruttando un'architettura distribuita basata su Apache Spark. Il sistema si basa su una topologia Master-Worker, dove il nodo Master agisce da orchestratore delle risorse, mentre i nodi Worker si occupano dell'esecuzione parallela dei task di calcolo.

Per semplicità, è stata adottata una strategia di Data Mirroring locale: i dataset e i modelli pre-addestrati sono stati replicati nella directory `/tmp` di ogni nodo, garantendo che ogni executor possa accedere alle risorse con latenza minima e **percorsi assoluti uniformi**, indipendentemente dalle specifiche configurazioni utente delle macchine fisiche. Tale sistema permette quindi la scalabilità orizzontale necessaria per l'elaborazione grandi quantità di dati. Seguono una serie di immagini che mostrano l'interfaccia web offerta da Spark, dove poter visualizzare i Task e i worker. Nel nostro caso abbiamo un worker che esegue i vari calcoli necessari alla visualizzazione delle funzioni descritte nei capitoli precedenti.



Spark Master at spark://192.168.0.151:7077

URL: spark://192.168.0.151:7077

Alive Workers: 1

Cores in use: 8 Total, 0 Used

Memory in use: 10.6 GiB Total, 0.0 B Used

Resources in use:

Applications: 0 [Running](#), 0 [Completed](#)

Drivers: 0 Running, 0 Completed

Status: ALIVE

Workers (1)

Worker Id	Address	State	Cores	Memory	Resources
worker-20260124141003-192.168.0.109-41021	192.168.0.109:41021	ALIVE	8 (0 Used)	10.6 GiB (0.0 B Used)	

Running Applications (0)

Application ID	Name	Cores	Memory per Executor	Resources Per Executor	Submitted Time	User	State	Duration
----------------	------	-------	---------------------	------------------------	----------------	------	-------	----------

Completed Applications (0)

Application ID	Name	Cores	Memory per Executor	Resources Per Executor	Submitted Time	User	State	Duration
----------------	------	-------	---------------------	------------------------	----------------	------	-------	----------



Spark Master at spark://192.168.0.151:7077

URL: spark://192.168.0.151:7077

Alive Workers: 1

Cores in use: 8 Total, 8 Used

Memory in use: 10.6 GiB Total, 1024.0 MiB Used

Resources in use:

Applications: 1 [Running](#), 0 [Completed](#)

Drivers: 0 Running, 0 Completed

Status: ALIVE

Workers (1)

Worker Id	Address	State	Cores	Memory	Resources
worker-20260124152458-192.168.0.109-44027	192.168.0.109:44027	ALIVE	8 (8 Used)	10.6 GiB (1024.0 MiB Used)	

Running Applications (1)

Application ID	Name	Cores	Memory per Executor	Resources Per Executor	Submitted Time	User	State	Duration
app-20260124152518-0000 (kill)	eRisk_Processor	8	1024.0 MiB		2026/01/24 15:25:18	luca	RUNNING	27 s

Completed Applications (0)

Application ID	Name	Cores	Memory per Executor	Resources Per Executor	Submitted Time	User	State	Duration
----------------	------	-------	---------------------	------------------------	----------------	------	-------	----------

Spark Worker at 192.168.0.109:44027

ID: worker-20260124152458-192.168.0.109-44027

Master URL: spark://192.168.0.151:7077

Cores: 8 (8 Used)

Memory: 10.6 GiB (1024.0 MiB Used)

Resources:

[Back to Master](#)

▼ Running Executors (1)

ExecutorID	State	Cores	Memory	Resources	Job Details	Logs
0	RUNNING	8	1024.0 MiB		ID: app-20260124152518-0000 Name: eRisk_Processor User: luca	stdout stderr

Application: eRisk_Processor

ID: app-20260124152518-0000

Name: eRisk_Processor

User: luca

Cores: Unlimited (8 granted)

Executor Limit: Unlimited (1 granted)

Executor Memory - Default Resource Profile: 1024.0 MiB

Executor Resources - Default Resource Profile:

Submit Date: 2026/01/24 15:25:18

State: RUNNING

[Application Detail UI](#)

▼ Executor Summary (1)

ExecutorID	Worker	Cores	Memory	Resource Profile Id	Resources	State	Logs
0	worker-20260124152458-192.168.0.109-44027	8	1024	0		RUNNING	stdout stderr

Capitolo 8

Conclusioni

Il presente progetto si inserisce nel filone di ricerca della salute mentale digitale, un campo in rapida espansione che vede i social media come "sensori sociali" capaci di fornire dati sul comportamento umano. Soluzioni analoghe, sono spesso sviluppate nell'ambito di competizioni internazionali come eRisk (da cui proviene il dataset utilizzato). L'approccio adottato in questo lavoro, combina l'analisi esplorativa psicolinguistica con modelli statistici.

8.1 Valutazione Tecnica

L'utilizzo di Apache Spark si è rivelato fondamentale per la gestione efficiente del dataset eRisk2018. La capacità di Spark di operare trasformazioni in memoria su oltre 600 000 righe, supportata dall'uso della cache, ha permesso di eseguire interrogazioni aggregate e analisi temporali complesse senza colli di bottiglia prestazionali.

8.2 Sviluppi Futuri

Il sistema presenta ampi margini di evoluzione tra cui:

- **Analisi del sentiment avanzato:** non limitarsi solo a identificare etichette come anoressia o depressione ma estendere l'analisi anche ad emozioni del tipo rabbia, tristezza, allegria.
- **Supporto Multilingua:** estendere i dataset a più lingue per rendere le varie interrogazioni più accurate.